2D Conduction in Composite Geometry

ME 541: Computational Heat Transfer

Daniel Kadyrov

May 19, 2016

Contents

1 Introduction **2**

2 Formulation **3**
2.1 Rectangular Domain . . . 4
2.2 Circular Domain . . . 5
2.3 Insulated Boundaries . . . 6
2.4 Contact Boundary . . . 7

3 Analysis **8**
3.1 Given Conditions . . . 8
3.2 Higher Amount of Point Values . . . 9
3.3 Change in Panel Dimensions . . . 10
3.4 Varying Initial Temperature Conditions . . . 11
3.5 Varying Boundary Temperature Conditions . . . 12
3.6 Equal K Values . . . 13

4 Conclusion **14**

5 Appendix **15**
5.1 MATLAB Code . . . 15

List of Figures

1 Geometry of Rectangular Composite . . . 2
2 Discretization of the Rectangular Domain . . . 4
3 Discretization of the Circular Domain . . . 5
4 Discretization of the Rectangular Domain with Insulated Boundaries . . . 6
5 Top Panel (Given Conditions) . . . 8
6 Bottom Panel (Given Conditions) . . . 8
7 Top Panel (Higher Amount of Points) . . . 9
8 Bottom Panel (Higher Amount of Points) . . . 9
9 Top Panel ($H_2 = 6$, $L = 5$) . . . 10
10 Bottom Panel ($H_1 = 2$, $L = 5$) . . . 10
11 Top Panel (Initial Temperature 400 °C) . . . 11
12 Bottom Panel (Initial Temperature 50 °C) . . . 11
13 Top Panel (Boundary Condition Temperature Change) . . . 12
14 Bottom Panel (Insulated Sides) . . . 12
15 Top Panel ($K = 4$) . . . 13
16 Bottom Panel ($K = 4$) . . . 13

List of Code Excerpts

1 Convergence Part 1 . . . 3
2 Convergence Part 2 . . . 3
3 Rectangular Section . . . 4
4 Semi-Circular Section . . . 5
5 Insulated Boundary Conditions . . . 6
6 Contact Boundary Conditions . . . 7

1 Introduction

Material composites are when two or more materials are combined for their physical and chemical properties. In some cases the materials are mixed, such as concrete, or one material is embedded in another, such as in fiberglass. In the most simple cases, these materials interact at their geometric boundaries and transfer various forms of energy between them. Figure 1 shows a simple geometry of a two-material rectangular composite.

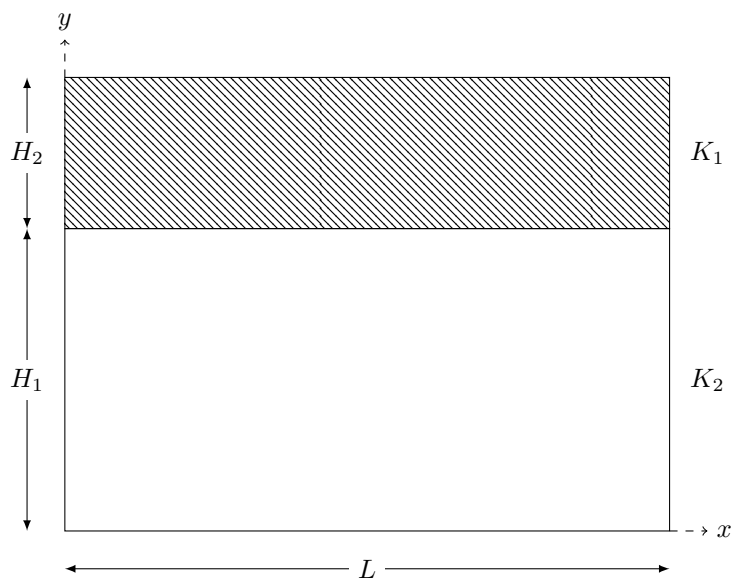


Figure 1: Geometry of Rectangular Composite

The temperature change within an object over time and space can be described by the Fourier transient heat conduction equation:

$$\kappa \nabla^2 T = \rho c_p \frac{\partial T}{\partial t} \tag{1}$$

where κ is the thermal conductivity of the material, ρ is its density, and c_p is its specific heat capacity. It is common to combine these material constants into one, K. Equation 1 then transforms to:

$$K \nabla^2 T = \frac{\partial T}{\partial t}$$

The solution to this equation requires an initial condition and boundary conditions. One common boundary condition is insulation, when the loss of heat, or heat flux, is prevented at a boundary.

$$\frac{\partial T}{\partial x} \mid_{x_{\text{Boundary}}} = 0 \tag{2}$$

2 Formulation

The analysis of the whole composite, with both panels and geometries, must be computed in separate parts, per panel and geometry, and then combined. The combination of both panels occurs at the contact boundary, where the temperature and heat flux must be equal. This contact boundary creates a equation that allows for the two temperature equations to be coupled.

Although it is possible to solve a solution of separate materials through creation of a space-changing K matrix, this method does not consider the change in geometries and possible change of coordinate systems.

The analysis of the problem is conducted until the steady state temperature occurs. Computationally this is solved by iterating a procedure until the difference in values between iterations reaches a tolerance level. Within MATLAB this is achieved using a **while** loop running the iterations until the tolerance is reached.

```
SteadyConvergence = 1e-4;
k = 0;
error = 1;

while ((error >= SteadyConvergence) && k < maxnt);
```

Code Excerpt 1: Convergence Part 1

```
  error = max(max(abs(T_1 - T_1_old)));
  k = k+1;
end
```

Code Excerpt 2: Convergence Part 2

2.1 Rectangular Domain

The rectangular domain is analyzed with Cartesian coordinates. In the Cartesian coordinate system the Fourier transient heat equation becomes:

$$K\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = \frac{\partial T}{\partial t} \tag{3}$$

In order to be evaluated computationally, the geometry of the figure is separated into nodes with a distance of Δx and Δy in the x and y directions, respectively. The distance between the nodes is determined by the desired accuracy of the analysis. At lower numbers the analysis yields more accurate results but it requires more computational power. The distance between the nodes inversely correlates with the total amount of nodes within the geometry. Figure 2 illustrates the mesh that results during this process.

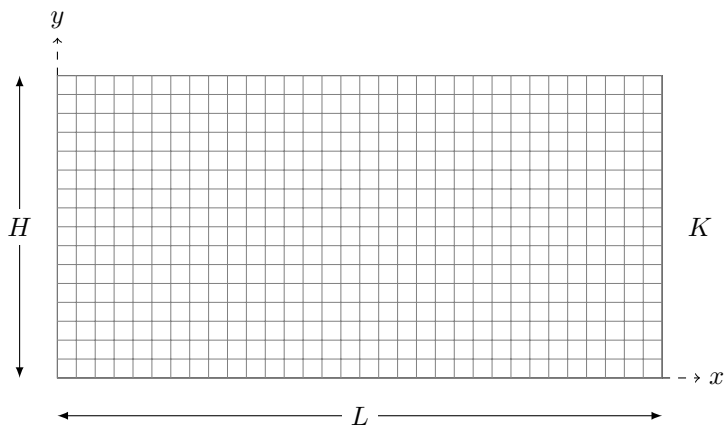


Figure 2: Discretization of the Rectangular Domain

The Forward-time Center Space (FTCS) scheme is a commonly used finite difference method to discretize the heat equation. Equation 4 shows the discretized form of the heat equation where i and j are the space steps for x and y, respectively, and n is the time step for t.

$$K\frac{T^n_{i+1,j} - 2T^n_{i,j} + T^n_{i+1,j}}{(\Delta x)^2} + K\frac{T^n_{i,j+1} - 2T^n_{i,j} + T^n_{i,j-1}}{(\Delta y)^2} = \frac{T^{n+1}_{i,j} - T^n_{i,j}}{\Delta t} \tag{4}$$

The equation can then solved for the next time step value, $T^{n+1}_{i,j}$, as shown below:

$$T^{n+1}_{i,j} = T^n_{i,j} + K\Delta t(\frac{T^n_{i+1,j} - 2T^n_{i,j} + T^n_{i+1,j}}{(\Delta x)^2} + \frac{T^n_{i,j+1} - 2T^n_{i,j} + T^n_{i,j-1}}{(\Delta y)^2})$$

Within MATLAB, this equation appears in 2 **for** loops, for x and y, respectively, for the inner section.

```
for ii = 2:nx-1;
  for jj = 2:ny-1;
    T_1(ii,jj) = T_1_old(ii,jj)+(dt*K1)*...
      ((T_1_old(ii+1,jj)-2*T_1_old(ii,jj)+T_1_old(ii-1,jj))/(dx^2)+ ...
      (T_1_old(ii,jj+1)-2*T_1_old(ii,jj)+T_1_old(ii,jj-1))/(dy^2));
  end
end
```

Code Excerpt 3: Rectangular Section

2.2 Circular Domain

Circular, or semi-circular, shapes are common geometric composites attached to squares or rectangles. Using polar coordinates makes analysis easier on these types of domains. In polar coordinates, the Fourier transient heat equation becomes:

$$K\frac{\partial^2 T}{\partial r^2} + K\frac{1}{r}\frac{\partial T}{\partial r} + K\frac{1}{r^2}\frac{\partial^2 T}{\partial \theta^2} = \frac{\partial T}{\partial t} \tag{5}$$

Figure 3 illustrates how this domain is discretized:

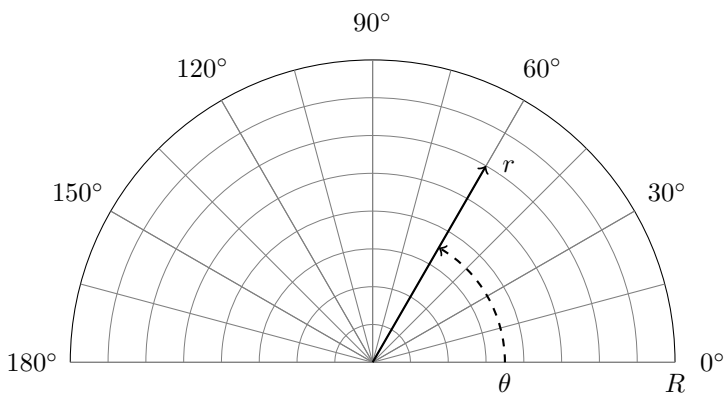


Figure 3: Discretization of the Circular Domain

Like in the rectangular section, the equation is discretized. Equation 6 shows the discretized form of the polar heat equation when solved for the future time step variable:

$$T_{r,\theta}^{n+1} = T_{r,\theta}^{n} + \Delta t K\left(\frac{T_{r+1,\theta}^{n} - 2T_{r,\theta}^{n} + T_{r-1,\theta}^{n}}{(\Delta r)^2} + \frac{1}{r}\frac{T_{r+1,\theta}^{n} - T_{r,\theta}^{n}}{\Delta r} + \frac{1}{r^2}\frac{T_{r,\theta+1}^{n} - 2T_{r,\theta}^{n} + T_{r,\theta-1}^{n}}{(\Delta\theta)^2}\right) \tag{6}$$

Within MATLAB, the temperature was calculated for the semi-circular section within 2 **for** loops:

```
for rr = 2:nr-1;
  for tO = 2:n_theta-1;
    T_p(rr,tO) = T_old0(rr,tO)+(dt/C)*...
      (((T_old0(rr+1,tO)-2*T_old0(rr,tO)+T_old0(rr-1,tO))/(dr^2))...
      *((1/rr)*(T_old0(rr+1,tO)-T_old0(rr-1,tO))/(dr^2))...
      *((1/rr^2)*(T_old0(rr,tO+1)-2*T_old0(rr,tO)+T_old0(rr,tO-1))/...
      (d_theta^2)));
  end
end
```

Code Excerpt 4: Semi-Circular Section

2.3 Insulated Boundaries

Insulation is when the loss of heat is prevented at a boundary. Mathematically, this is expressed in Equation 2. Figure 4 shows the rectangular domain with insulated boundaries on the x boundary conditions.

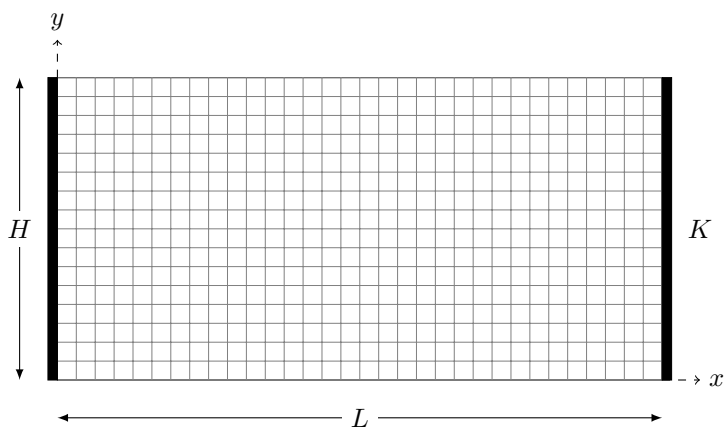


Figure 4: Discretization of the Rectangular Domain with Insulated Boundaries

The insulated boundary conditions are discretized the following way:

$$T^n_{0,j} = T^n_{1,j}$$
$$T^n_{nx,j} = T^n_{nx-1,j}$$

Within MATLAB, this appears as:

```
for jj = 2:ny-1;
  for ii = 1; % at x = 0
    T_1(ii,jj) = T_1_old(ii,jj)+(dt*K1)*...
      (((T_1_old(ii+1,jj)-2*T_1_old(ii+1,jj)+T_1_old(ii+1,jj))/(dx^2))*...
      ((T_1_old(ii+1,jj+1)-2*T_1_old(ii+1,jj)+T_1_old(ii+1,jj-1))/(dy^2)));
  end
  for ii = nx; % at x = nx
  T_1(ii,jj) = T_1_old(ii,jj)+(dt*K1)*...
    (((T_1_old(ii-1,jj)-2*T_1_old(ii-1,jj)+T_1_old(ii-1,jj))/(dx^2))*...
    ((T_1_old(ii-1,jj+1)-2*T_1_old(ii-1,jj)+T_1_old(ii-1,jj-1))/(dy^2)));
  end
end
```

Code Excerpt 5: Insulated Boundary Conditions

2.4 Contact Boundary

The contact boundary is the point at which two geometries or different materials are in contact. At this point, not only should the temperature be the same, but so should the heat flux between the two materials. In the example given in Figure 1, the contact point is when $y = H_1$. Mathematically, the temperature and heat flux at this contact boundary can be expressed the following way:

$$T_1(x, H_1) = T_2(x, 0) \tag{7}$$

$$K_1 \frac{\partial T_1}{\partial y_1} \mid_{H_1} = K_2 \frac{\partial T_2}{\partial y_2} \mid_0 \tag{8}$$

These equations are discretized to the following form:

$$T^1_{i,m} = T^2_{i',0} \tag{9}$$

$$K_1 \frac{T^1_{i,m} - T^1_{i,m-1}}{\Delta y_1} = K_2 \frac{T^2_{i',1} - T^2_{i',0}}{\Delta y_2} \tag{10}$$

Substituting Equation 9 into Equation 10 and solving for the value of the contact point gives:

$$T^1_{i,my} = T^2_{i',0} = \frac{K_1 \Delta y_2 T^1_{i,my-1} + K_2 \Delta y_1 T^2_{i',1}}{K_1 \Delta y_2 + K_2 \Delta y_1}$$

This definition can be used with other geometries, including polar coordinates. For example, the following equation expresses the contact boundary between a rectangle and a semi-circle at the top boundary.

$$T^1_{i,my} = T^2_{r,0} = \frac{K_1 \Delta \theta T^1_{i,my-1} + K_2 \Delta y T^2_{r,1}}{K_1 \Delta \theta + K_2 \Delta y_1}$$

Within MATLAB, this appears as:

```
for jj = ny;
  for jj1 = 0;
    for ii = 1:nx-1;
      for ii1 = 1:nx_1;
        T_1(ii,jj) = (K1*dy1*T_1_old(ii,jj-1)+...
          K2*dy*T_2_old(ii1,jj1+1))/(K1*dy1+K2*dy);
        T_2(ii1,jj1+1) = T_1(ii,jj);
        T_1(ii,jj) = T_2(ii1,jj1+1);
      end
    end
  end
end
```

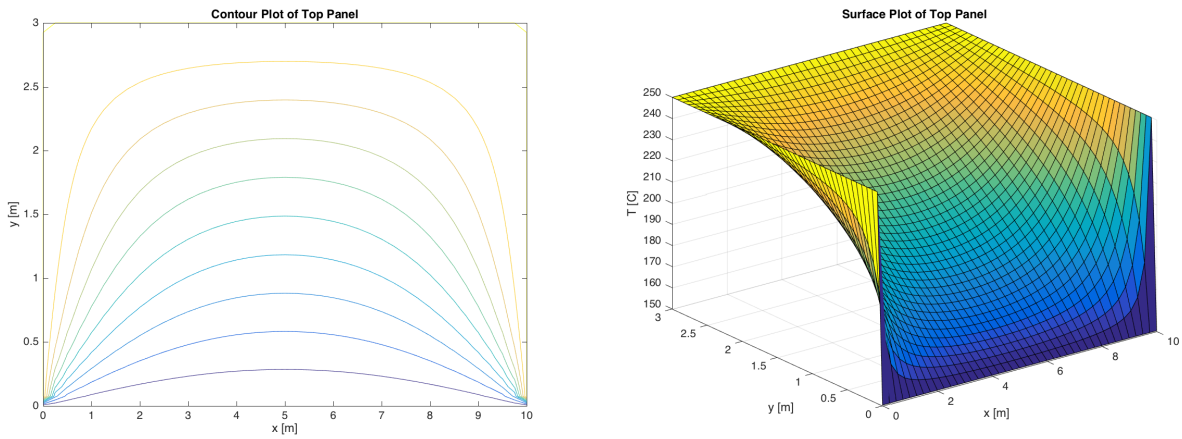
Code Excerpt 6: Contact Boundary Conditions

3 Analysis

Analysis was conducted of the problem shown in Figure 1. Although the derivation of a circular geometry was done, MATLAB encounters errors dealing with polar coordinates and releases inaccurate results. The analysis of MATLAB code was conducted on the following varying conditions: given conditions, higher amount of point values, varying dimension sizes, varying initial conditions, varying boundary temperature conditions, and equal K values.

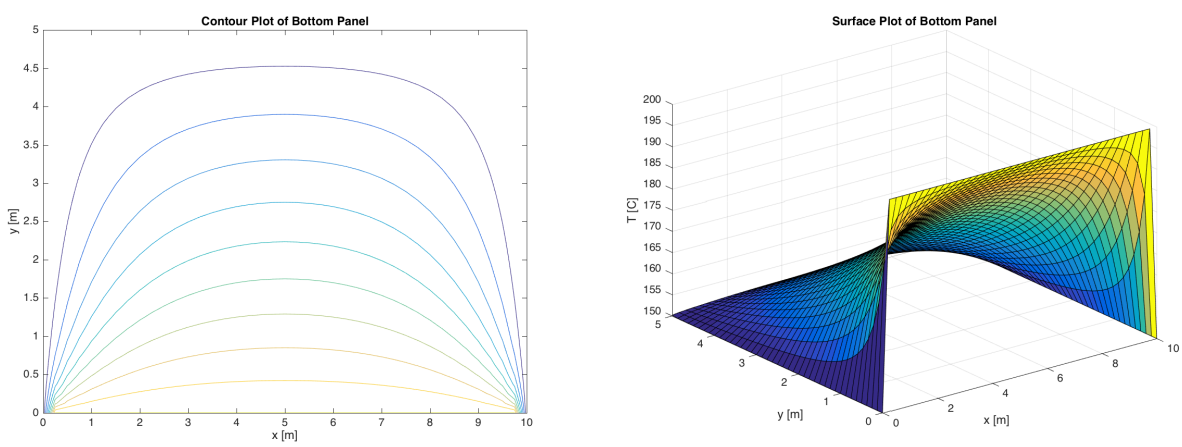
3.1 Given Conditions

The lower panel has a length, L, of 10 meters, a height, H_1, of 5 meter, and the value of K_1 is equal to 5. The upper panel has the same length but the height, H_2 is 3 meters and the value of K_2 is equal to 3. Both panels have an initial temperature condition of 100 °C. The lower panel has a constant temperature of 200 °C on the bottom boundary and a constant temperature of 150 °C on the sides. The top panel has constant temperature of 250 °C on all three sides. The number of points used in the analysis was 40.



(a) Contour Plot (b) Surface Plot

Figure 5: Top Panel (Given Conditions)

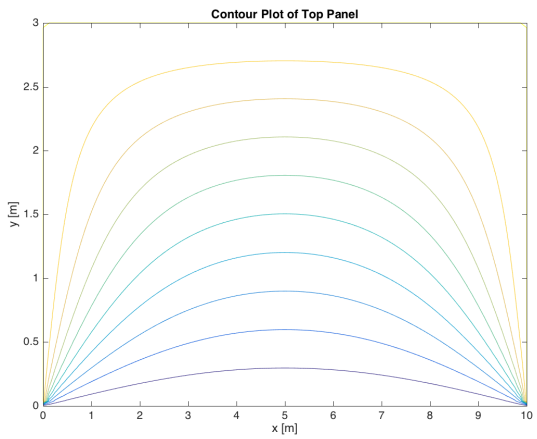


(a) Contour Plot (b) Surface Plot

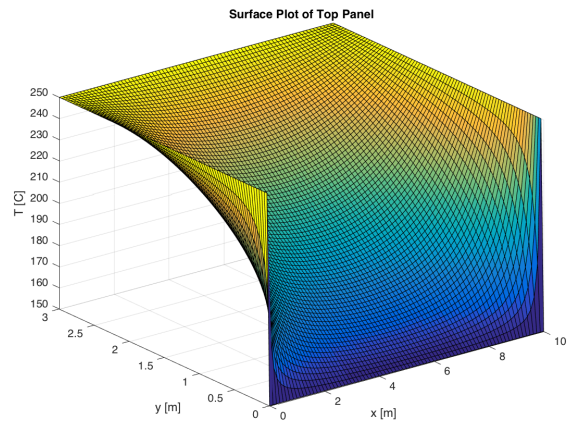
Figure 6: Bottom Panel (Given Conditions)

3.2 Higher Amount of Point Values

The problem was then analyzed with 80 points.

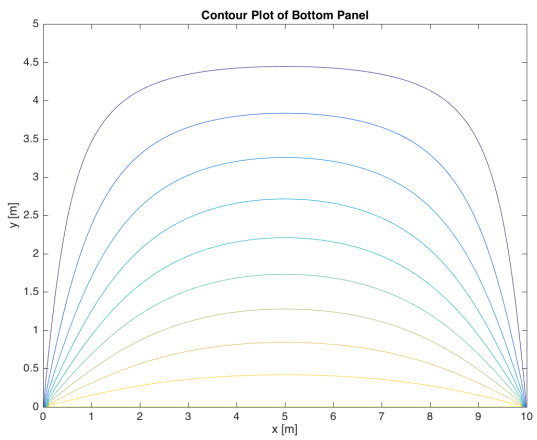


(a) Contour Plot

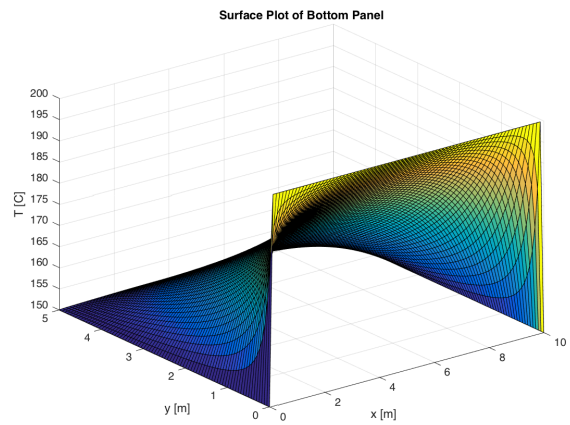


(b) Surface Plot

Figure 7: Top Panel (Higher Amount of Points)



(a) Contour Plot

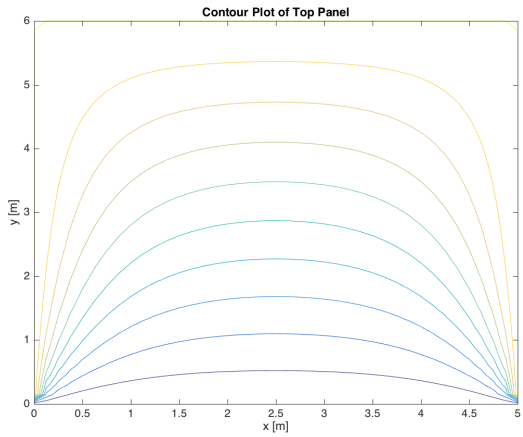


(b) Surface Plot

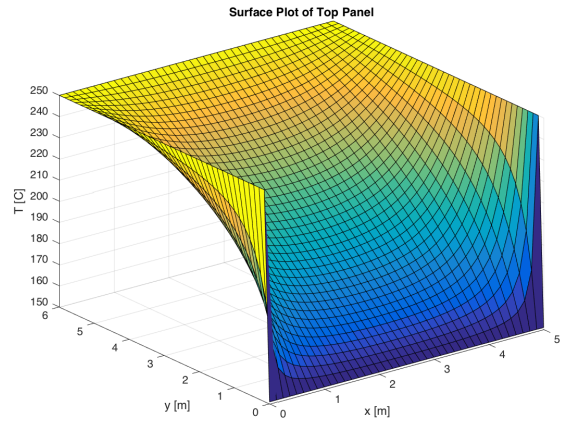
Figure 8: Bottom Panel (Higher Amount of Points)

3.3 Change in Panel Dimensions

The problem was then analyzed with the bottom panel height, H_1, changed to 2 meters and the top panel height, H_2, changed to 6 meters. The length, L, for both panels was changed to 5 meters.

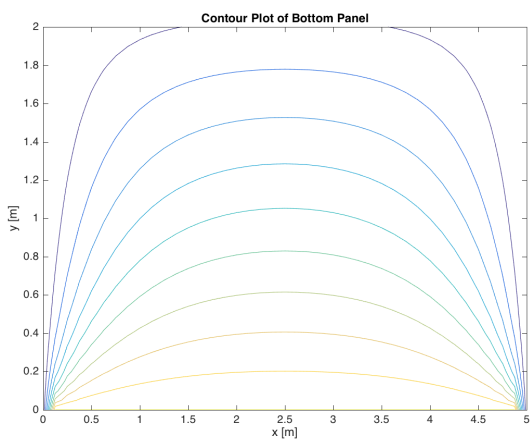


(a) Contour Plot

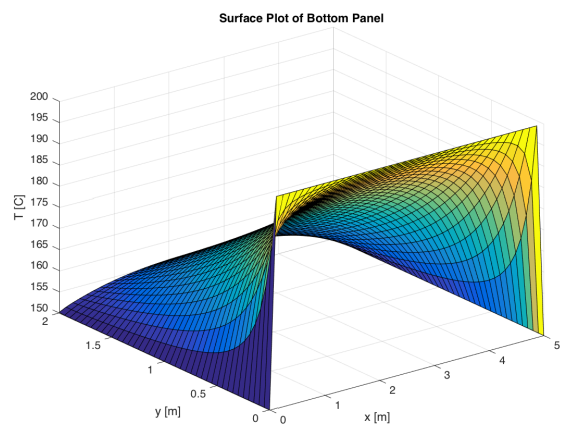


(b) Surface Plot

Figure 9: Top Panel ($H_2 = 6$, $L = 5$)



(a) Contour Plot

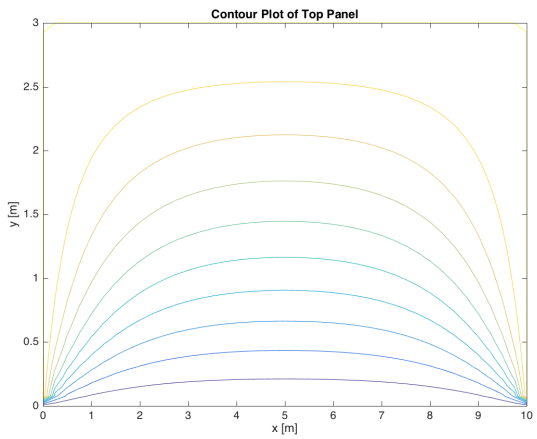


(b) Surface Plot

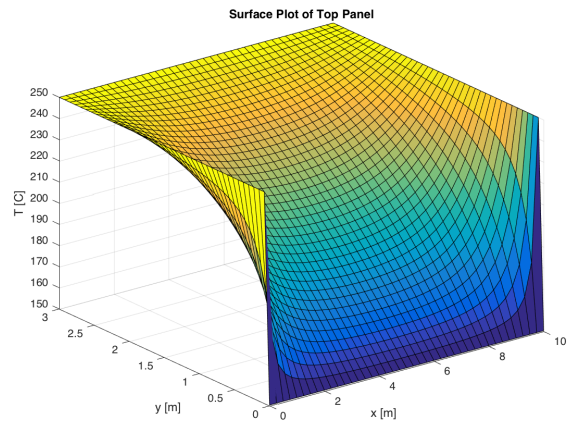
Figure 10: Bottom Panel ($H_1 = 2$, $L = 5$)

3.4 Varying Initial Temperature Conditions

The initial condition of the lower panel was changed to 50 °C and to 400 °C for the top panel.

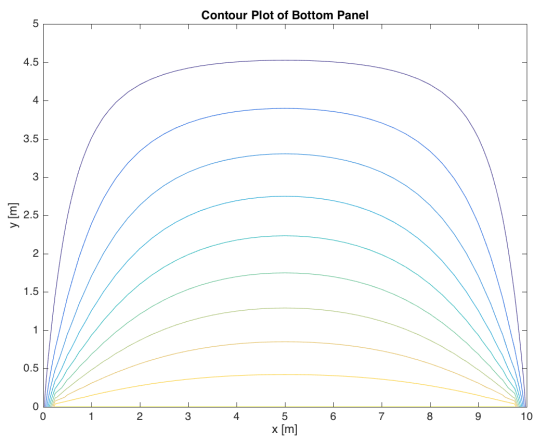


(a) Contour Plot

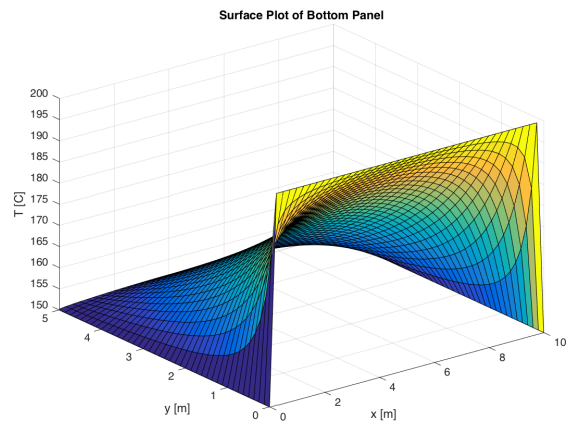


(b) Surface Plot

Figure 11: Top Panel (Initial Temperature 400 °C)



(a) Contour Plot

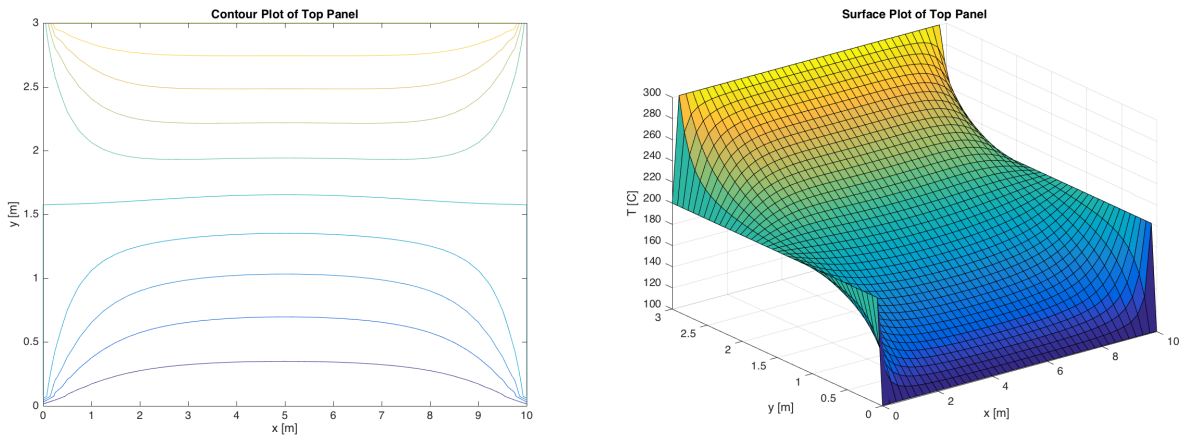


(b) Surface Plot

Figure 12: Bottom Panel (Initial Temperature 50 °C)

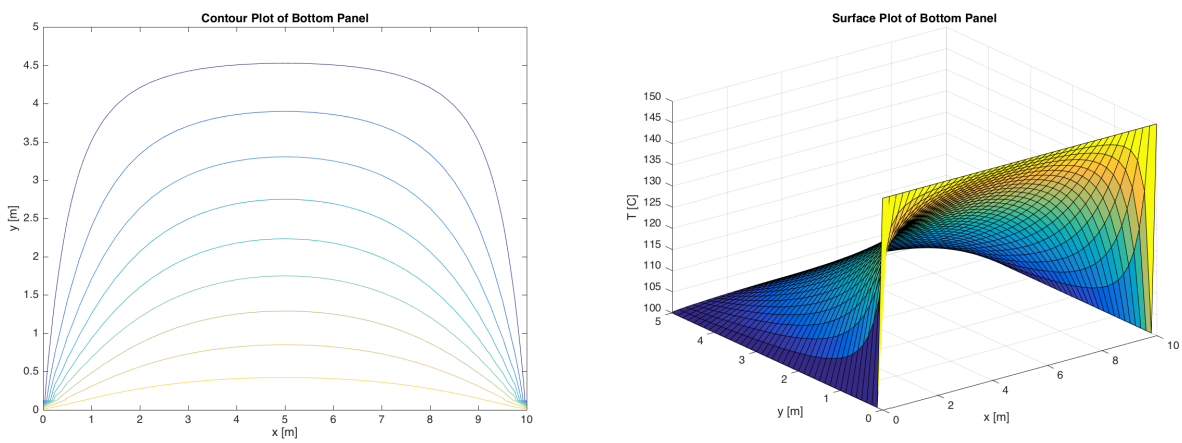
3.5 Varying Boundary Temperature Conditions

The boundary conditions for the top panel were changed to a constant temperature of 200 °C on the sides and 300 °C at the top boundary. The bottom panel's sides were changed to have insulated conditions and a constant 150 °C at the lower boundary.



(a) Contour Plot (b) Surface Plot

Figure 13: Top Panel (Boundary Condition Temperature Change)

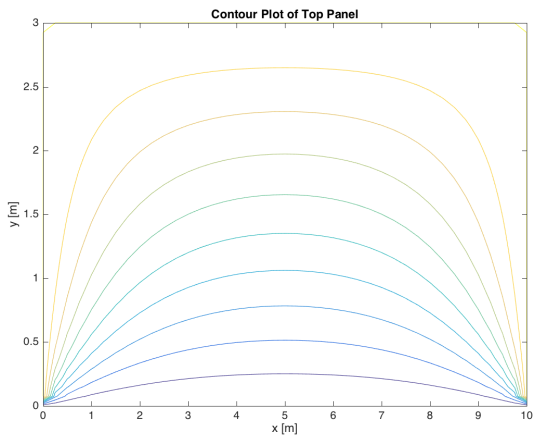


(a) Contour Plot (b) Surface Plot

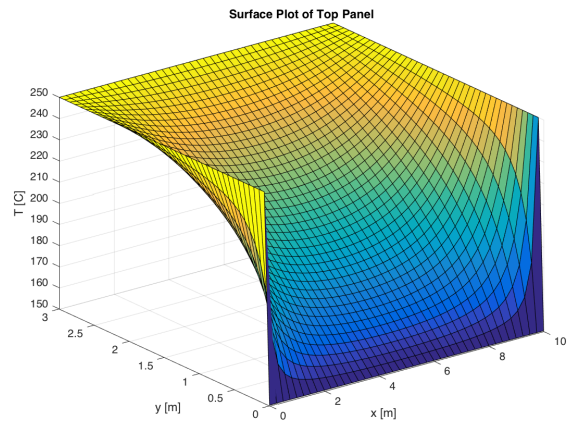
Figure 14: Bottom Panel (Insulated Sides)

3.6 Equal K Values

The K values were made equal and set to 4.

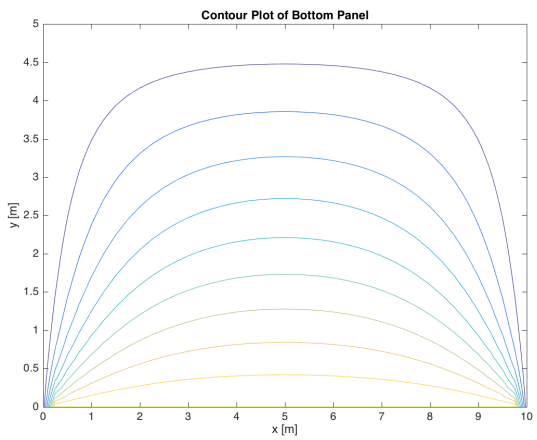


(a) Contour Plot

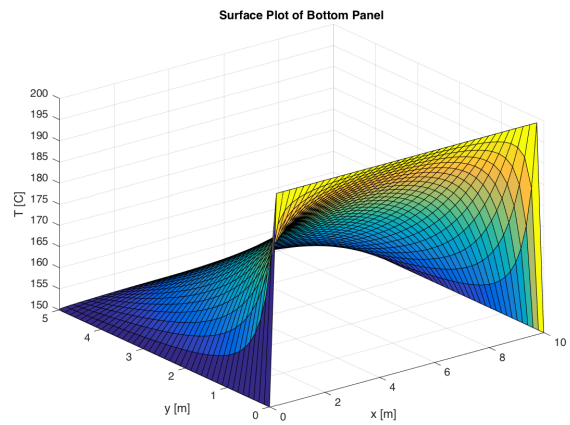


(b) Surface Plot

Figure 15: Top Panel ($K = 4$)



(a) Contour Plot



(b) Surface Plot

Figure 16: Bottom Panel ($K = 4$)

4 Conclusion

Material composites occur in a variety of applications with varying constants and geometries. In order to solve these problems computationally, the contact boundary between the two geometries needs to be taken into consideration.

The method used in this problem was demonstrated through a variety of parameter changes including using higher amount of node points, changing the dimensions of the geometry, varying the initial conditions, varying the boundary conditions, including insulated boundaries, and changing the constant value of K.

Changing the amount of nodes used to a higher number allows more accurate results. This however requires more computation power from the computer and therefore takes longer to converge. Changing the dimensions of the geometry changed the results of the computation and verified that the model continued to work for smaller heights and lengths. As the dimensions decreased but the temperature at the boundaries stayed the same, the effects of the conduction were more prevalent on the panels.

Varying the initial conditions did not change the results since analysis was only done on the steady state converged time. If the results did change, this would indicate an issue in the model. The boundary conditions were changed to different values and some were selected to be insulated. These affected the model results accordingly. Changing the value of K to be equal caused the panels, and the materials they were made of, to behave the same way under the steady temperatures.

Overall this model works accurately and quickly. Using MATLAB for computational modeling comes with limitations including inability to use polar coordinates and lack of computational power. It can be seen that with higher amount of nodes, the results are more accurate however it is unrealistic to use high numbers if it takes extensively long periods of time for the program to run. Using a faster computer and a more powerful language, such as Python, would allow for better analysis for this type of problem. Software such as FLUENT, which uses complex finite-element algorithms to solve heat transfer problems, can also successfully approach complicated geometries.

5 Appendix

5.1 MATLAB Code

```
% Daniel Kadyrov
% 2D Conduction in Composite Geometry

close all
clear all
clc

% Define Constants
L = 10; % Length [m]
H1 = 5; % Height 1 [m]
H2 = 3; % Height 2 [m]
K1 = 5; % K for Bottom Panel
K2 = 3; % K for Top Panel

% Number of Points for Analysis
invx = 40;
invy = 40;
invx1 = 40;
invy1 = 40;

% Time Conditions
dt = 0.0001; % Number of time values
t = 0:dt:100;
maxnt = length(t);

% Rectangular Conditions
dx = L/invx;
dy = H1/invy;
dx1 = L/invx1;
dy1 = H2/invy1;
x = 0:dx:L;
y = 0:dy:H1;
x1 = 0:dx1:L;
y1 = 0:dy1:H2;

nx = length(x);
ny = length(y);
nx_1 = length(x1);
ny_1 = length(y1);

T_1 = zeros(nx);
T_2 = zeros(nx_1);

% Initial Conditions
T_1(:,:) = 100; % Lower Panel
T_2(:,:) = 100; % Top Panel

% Boundary Conditions for Lower Panel
T_1(:,1) = 200; % Bottom Boundary
T_1(1,:) = 150; % Left Boundary
T_1(nx,:) = 150; %Right Boundary
```

```
% Boundary Conditions for Top Panel
T_2(:,ny_1) = 250; % Upper Boundary
T_2(1,:) = 250; % Left Boundary
T_2(nx_1,:) = 250; % Right Boundary

% Convergence Criteria
SteadyConvergence = 1e-4;
k = 0;
error = 1;

while ((error >= SteadyConvergence) && k < maxnt);
    T_1_old = T_1;
    T_2_old = T_2;

    % Bottom Panel
    for ii = 2:nx-1;
        for jj = 2:ny-1;
            T_1(ii,jj) = T_1_old(ii,jj)+(dt*K1)*...
                ((T_1_old(ii+1,jj)-2*T_1_old(ii,jj)+T_1_old(ii-1,jj))/
(dx^2)+ ...
                (T_1_old(ii,jj+1)-2*T_1_old(ii,jj)+T_1_old(ii,jj-1))/
(dy^2));
        end
    end

    % Top Panel
    for ii = 2:nx_1-1;
        for jj = 2:ny_1-1;
            T_2(ii,jj) = T_2_old(ii,jj)+(dt*K2)*...
                ((T_2_old(ii+1,jj)-2*T_2_old(ii,jj)+T_2_old(ii-1,jj))/
(dx^2)+ ...
                (T_2_old(ii,jj+1)-2*T_2_old(ii,jj)+T_2_old(ii,jj-1))/
(dy^2));
        end
    end

    % Contact Point Boundary
    for jj = ny;
        for jj1 = 0;
            for ii = 1:nx-1;
                for ii1 = 1:nx_1;
                    T_1(ii,jj) =
 (K1*dy1*T_1_old(ii,jj-1)+K2*dy*T_2_old(ii1,jj1+1))/(K1*dy1+K2*dy);
                    T_2(ii1,jj1+1) = T_1(ii,jj);
                    T_1(ii,jj) = T_2(ii1,jj1+1);
                end
            end
        end
    end

    % Insulated Boundary Condition for Bottom Panel
%     for jj = 2:ny-1;
%         for ii = 1; % at x = 0
%             T_1(ii,jj) = T_1_old(ii,jj)+(dt*K1)*...
```

```
%                 (((T_1_old(ii+1,jj)-2*T_1_old(ii+1,jj)+T_1_old(ii
+1,jj))/(dx^2))*...
%                 ((T_1_old(ii+1,jj+1)-2*T_1_old(ii+1,jj)+T_1_old(ii
+1,jj-1))/(dy^2)));
%         end
%         for ii = nx; % at x = nx
%             T_1(ii,jj) = T_1_old(ii,jj)+(dt*K1)*...
%
 (((T_1_old(ii-1,jj)-2*T_1_old(ii-1,jj)+T_1_old(ii-1,jj))/(dx^2))*...
%                 ((T_1_old(ii-1,jj
+1)-2*T_1_old(ii-1,jj)+T_1_old(ii-1,jj-1))/(dy^2)));
%         end
%     end

    error = max(max(abs(T_1 - T_1_old)));
    k = k+1;
end

T_1 = T_1';
[L,H] = meshgrid(x,y);

figure(1)
contour(L,H,T_1);
title('Contour Plot of Bottom Panel')
xlabel('x [m]')
ylabel('y [m]')

figure(2)
surf(x,y,T_1);
xlabel('x [m]')
ylabel('y [m]')
zlabel('T [C]')
title('Surface Plot of Bottom Panel')

T_2 = T_2';
[L1,H1] = meshgrid(x1,y1);

figure(3)
contour(L1,H1,T_2);
title('Contour Plot of Top Panel')
xlabel('x [m]')
ylabel('y [m]')

figure(4)
surf(x1,y1,T_2);
xlabel('x [m]')
ylabel('y [m]')
zlabel('T [C]')
title('Surface Plot of Top Panel')
```